

Full Demo Playbook — Viva Presentation

Memory-Based Detection of Golden Ticket & SIDHistory Abuse in Active Directory

P Rahul · R23MTC09. Follow top to bottom. Each step: **RUN** (what to do) · **SHOW** (what's on screen) · **SAY** (your line).

PART 0 — Before the viva (do 20-30 min ahead)

0.1 — Reboot the DC VM first (fixes AMA log ingestion).

The DC VMware VM must be freshly booted and set to NOT suspend, or the log-based Golden Ticket path won't ingest. Reboot it, wait 5 min.

0.2 — Bring the lab up. Double-click `C:\SecurityScripts\START-LAB.cmd`.

Confirm it ends with "lab is UP" and prints the dashboard URL. It self-heals AMA.

0.3 — Verify ingestion is live. In Sentinel → Logs run:

`SecurityEvent | summarize max(TimeGenerated)` → must be < 5 min old. If stale, wait or reboot DC again.

0.4 — Pre-run the attack ONCE (so incidents already exist — never wait live in the room).

Run the three attack steps (Golden Ticket + SIDHistory + LSASS capture). Note the new incident numbers. Detection takes ~5-10 min, so doing this ahead means finished evidence is ready.

0.5 — Open these tabs in advance:

- Azure portal → Sentinel → **HybridDetectionWS** (correct tenant: agpandian@hotmail.com)
 - Sentinel → Incidents (Last 24h filter)
 - Live dashboard: <https://capstonedash824f.z13.web.core.windows.net/>
 - Both workbooks (Corroboration + Evidence)
 - A terminal ready with the attack command (for the live "act")
 - **Screenshots folder as fallback** (in case portal/VM misbehaves)
-

PART 1 — The live demo (target ~6-8 min)

STEP 1 — Frame the problem (30 sec, no screen)

SAY: "Golden Ticket and SIDHistory are among the stealthiest AD attacks — the attacker becomes a legitimate-looking admin. Log-only SIEM can be evaded, and it never sees the forged ticket itself, which sits in the Domain Controller's memory. I built a hybrid framework that detects both attacks AND recovers the forged ticket from memory — corroborating the logs with the actual artifact."

STEP 2 — Show the architecture (1 min)

RUN: open `architecture-diagram.html` in a browser.

SHOW: the three detection paths — Kerberos logs, SIDHistory inventory, LSASS memory forensics.

SAY: "Events flow from the DC via Azure Arc and the Monitor Agent into Sentinel. In parallel,

LSASS memory is captured, analysed on a Volatility VM, and the result flows back to a custom table. Three independent detection paths converge into Sentinel incidents."

STEP 3 — Run the attack LIVE (1 min) — the "act"

RUN (in your ready terminal): the Golden Ticket command — DCSync to steal the krbtgt hash, forge a ticket for Administrator, inject it (Pass-the-Ticket), and trigger the LSASS capture.

SHOW: mimikatz output; `10/10 TGS retrieved`; "ticket resident: True".

SAY: "I've just stolen the krbtgt key, forged a Golden Ticket for Administrator, injected it into memory, and triggered a memory capture. Now let's see the framework catch it — two ways."

Then CUT to the incidents you pre-ran in Step 0.4 — do NOT wait 10 min live.

STEP 4 — Show the incidents (1 min) — detection fired

RUN: Sentinel → Incidents (Last 24h).

SHOW: the three types, all High:

- Golden Ticket - TGS Without Prior TGT
- SIDHistory - New Entry Delta Detection
- **Volatility Memory Analysis - HIGH/CRITICAL** (the memory one)

SAY: "Within minutes Sentinel raised incidents on all three paths, auto-triaged and labelled."

STEP 5 — The recovered artifact (1.5 min) — YOUR DIFFERENTIATOR

RUN: Sentinel → Logs → **Queries** tab → group "**Capstone Evidence**" → double-click "**01 - Memory: Recovered Forged Ticket**" → Run.

SHOW: the row — RecoveredTicketFile (`TGT..._Administrator_krbtgt...kirbi`), TargetAccount Administrator, **LifetimeDays 3650**, tickets extracted.

SAY: "This is the difference. A log-only SIEM tells you an attack *probably* happened. Here is the **actual forged ticket recovered from the DC's memory** — for Administrator, with a 3650-day, ten-year lifetime. A real ticket lasts ~10 hours; ten years is the forged signature. That's the smoking gun, not an inference."

STEP 6 — Corroboration (1 min) — the thesis proof

RUN: Sentinel → Workbooks → "**Attack Corroboration Dashboard (MTech)**" → View.

SHOW: the top panel — **CONFIRMED - Log + Memory**, Administrator, Log count + Memory count, 3650-day lifetime; then the two tracks side by side.

SAY: "The same attack, on the same account, confirmed by two independent methods — the log track and the memory track. An inference *and* the artifact. That corroboration is the core contribution."

STEP 7 — The live dashboard (30 sec) — the showpiece

RUN: open <https://capstonedash824f.z13.web.core.windows.net/> and full-screen (F11).

SHOW: KPI tiles, corroboration verdict, recovered-ticket panel, MITRE, benchmark — all live.

SAY: "Everything at a glance — hosted in Azure, auto-updating from Sentinel via a managed identity. This stays live even when the lab VM is off."

STEP 8 — Results & MITRE (30 sec)

RUN: either the Evidence workbook or Sentinel → MITRE ATT&CK.

SHOW: 16 rules, MITRE techniques T1558 / T1550 / T1134 / T1003 lit up; the benchmark panel (SIDHistory detector: Precision/Recall/F1 = 1.00 on 40 labelled samples).

SAY: "16 analytics rules mapped to MITRE. For the SIDHistory detector I ran a labelled benchmark — precision, recall and F1 of 1.00 on 40 samples. Detection time dropped from a ~45-minute baseline to a few minutes."

PART 2 — Fallback plan (if something breaks live)

If this breaks	Do this
--- ---	
Portal slow / wrong tenant	Present from the screenshots folder; say "captured live on <date>"
AMA / log path down	Lead with memory + SIDHistory (they work without AMA) + the live dashboard; note log path validated previously (incidents #472-477)
VM asleep / worker down	Show the live dashboard + workbooks (they read stored data, work with VM off)
Attack command errors live	Skip the live act; go straight to the pre-run incidents (Step 4)
Workbook slow to load	Use the saved queries (Logs → Queries → Capstone Evidence) instead

Golden rule: never debug live in front of examiners. Switch to screenshots and keep talking.

PART 3 — Cheat sheet

URLs

- Live dashboard: <https://capstonedash824f.z13.web.core.windows.net/>
- Portal: portal.azure.com → tenant **agpandian@hotmail.com** → Sentinel → **HybridDetectionWS**

Where each output lives

Output	Location
--- ---	
Incidents	Sentinel → Incidents (Last 24h)
Recovered ticket (artifact)	Logs → Queries → "Capstone Evidence" → 01
Corroboration	Workbook: Attack Corroboration Dashboard (MTech)
Overview	Workbook: Capstone Evidence (MTech)
Live showpiece	The dashboard URL
MITRE	Sentinel → MITRE ATT&CK

Key numbers to have ready

- Ticket lifetime: **3650 days (~10 years)** — the forged signature
- SIDHistory benchmark: **Precision/Recall/F1 = 1.00** (40 labelled samples)
- Rules: **16**, mapped to MITRE
- Detection time: **~45 min baseline → a few minutes**
- 3 detection paths: Kerberos logs, SIDHistory inventory, LSASS memory forensics

One-line summary (memorise):

"Every existing tool stops at an inference or needs the attacker's endpoint. I built the missing piece — DC-side memory forensics that recovers the actual forged ticket — and corroborated it with the log track. 'Probably an attack' becomes 'here's the weapon, confirmed two independent ways.'"

Honest answers to keep ready

- *Which memory tool?* → "Custom LSASS memory analysis on the Volatility VM; standard Volatility 3 has no Kerberos-ticket plugin on modern Windows, so I use a memory-parsing approach that does."
 - *Accuracy?* → "Per-case validation + a measured SIDHistory benchmark (P/R/F1 = 1.00, 40 samples). No single blanket accuracy — a larger adversarial dataset is future work."
 - *Golden vs real ticket?* → "Ticket lifetime — forged ~10 years, real ~10 hours — plus TGS with no prior TGT in the logs."
 - *Limitations?* → "Single-DC lab; capture timing matters; automated incident-triggered capture and a larger benchmark are future work."
-

PART 4 — After the demo

Run `C:\SecurityScripts\STOP-LAB.cmd` to save credits (the live dashboard stays up).